

Scilab Cheatsheet

A quick reference distilled from *Scilab for Very Beginners*. Scilab is numerical computation software — everything is a number (or a matrix of numbers). It does **not** do symbolic algebra.

The Basics

Concept	Notes
Prompt	Type a command after <code>--></code> and press Enter
<code>ans</code>	Auto-stores any result you don't assign to a variable
<code>;</code> (semicolon)	Suppresses output — calculation runs, nothing prints
<code>//</code>	Comment — ignored by the calculator
Case-sensitive	<code>sqrt(9)</code> works, <code>SQRT(9)</code> is an error
Tab key	Auto-completes a function/variable name from its first letters
<code>..</code> (two dots)	Continues a long command onto the next line

Arithmetic operators

`+` `-` `*` `/` `^` (exponent)

Built-in constants

Symbol	Value
<code>%e</code>	$e \approx 2.7182818$
<code>%pi</code>	$\pi \approx 3.1415927$
<code>%i</code>	imaginary unit i
<code>%eps</code>	smallest relative precision $\approx 2.22e-16$ (~16 digits)
<code>%T</code> / <code>%F</code>	true / false

Variables & Functions

```
scilab

a = %pi/4      // assignment
Piby2 = %pi/2  // any name not already used by the system
```

Variables don't need declaring, but using an undefined one throws an error. Names can't have accents or special characters.

Defining a function

```
scilab

function y=f(x); y=36/(8+exp(-x)); endfunction
f(10)           // call it

// multi-line version in the editor:
function y=f(x)
    y = (x^2 + 2*x)*exp(-x)
endfunction
```

The variable names inside (x), (y) are local/dummy — you can reuse them elsewhere.

Vectors & Matrices

Square brackets define them. **Space or comma** separates columns; **semicolon** separates rows.

```
scilab

v = [3, -2, 5]    // row vector
v = [3; -2; 5]    // column vector
m = [1 2 3; 4 5 6] // 2x3 matrix
```

Accessing & editing elements

```
scilab
```

```
m(2,3)    // element at row 2, col 3
m(2,3) = 23 // change it
m(2,:)    // entire 2nd row
m(:,3)    // entire 3rd column
m'        // transpose (single quote)
```

The colon (:) operator (ranges)

Format is `start:step:end`. Step defaults to 1 if omitted.

```
scilab

3:10      // 3 4 5 6 7 8 9 10
1:2:10    // 1 3 5 7 9
20:-4:2   // 20 16 12 8 4
```

Matrix vs. element-wise operations

`*` and `/` are **matrix** operations. Add a **dot** for element-wise:

```
scilab

A*B        // matrix multiplication (dimensions must match)
A.*A       // element-wise multiply
A./A       // element-wise divide
C^2        // for vectors, exponent IS element-wise
```

Solving a linear system $AX = Y$

```
scilab

X = A \ Y  // backslash; A must be square-ish for the system
           // note: Y/A solves a DIFFERENT system (ZA = Y)
```

Loops

`for` — fixed number of iterations

```
scilab
```

```

u(1) = 4;
for n = 1:20
    u(n+1) = u(n) + 2*n + 3;
    disp([n, u(n)])
end

```

while — loop until a condition is met

```

scilab

h = 1.2; y = 2005;
while h < 7
    h = h + 0.3;
    y = y + 1;
end
disp("Cut the tree in " + string(y))

```

Tests & Conditionals

Comparison / logical operators

Equal	Not equal	Less	Greater	≤	≥	And	Or	Not
<code>==</code>	<code><></code>	<code><</code>	<code>></code>	<code><=</code>	<code>>=</code>	<code>&</code>	<code> </code>	<code>~</code>

Comparing vectors with `==` works term-by-term. To test if two whole vectors match, use `isequal(X,Y)`.

if / elseif / else

`if ... then` must be on one line (same for `elseif ... then`).

```

scilab

```

```

if D == [6,6,6] then
    W = 20;
elseif length(unique(D)) == 1 then
    W = 10;
elseif length(unique(D)) == 2 then
    W = 5;
else
    W = 0;
end

```

Accuracy warning: calculations are approximate, so `==` can give false negatives. `sin(%pi)` returns $\sim 1.2\text{e-}16$, not exactly 0. Compare against `%eps` instead of testing exact equality, and prefer **relative** precision: `abs(a^2+b^2-c^2)/c^2 < %eps`.

Plotting

2-D

```

scilab

plot(1, 2, "r")      // single red point
plot([1,3], [2,5])   // segment between two points
x = linspace(-2, 5, 50); // 50 evenly spaced x-values
plot(x, f, "r", x, g, "g") // two curves, red and green
plot(u, "*r")        // sequence of points as red stars
clf                  // clear figure

```

`linspace(a, b, n)` = vector of `n` values evenly spaced from `a` to `b`.

Colors & styles (inside quotes)

- Colors: `"b"` blue (default), `"k"` black, `"r"` red, `"g"` green, `"c"` cyan, `"m"` magenta, `"y"` yellow, `"w"` white
- Point styles: joined (default), `"."`, `"+"`, `"o"`, `"x"`, `"*"`, `"s"`, `"d"`, `"v"`, `"<"`, `">"`

Bar graphs

```

scilab

bar(x, n)           // single bar graph
bar(X, [n1', n2'])  // side-by-side (n1, n2 as column vectors)

```

3-D

scilab

```
// Surface z = f(x,y)
function z=f(x,y); z = 2*x^2 + y^2; endfunction
x = linspace(-1,1,100);
y = linspace(-2,2,200);
z = feval(x,y,f)';      // note the transpose '
surf(x, y, z)

// Space curve (e.g. a helix)
t = linspace(0, 4*%pi, 100);
param3d(cos(t), sin(t), t)
```

Managing windows

- `clf` — clear current figure
 - `scf;` — open/set a new graphics window ("set current figure")
 - `scf(n);` — switch to graphics window number `n`
-

Differential Equations

Reduce any explicit system to first order, then use `ode`:

scilab

```
y = ode(y0, t0, t, f)
```

- `y0` initial condition (vector)
- `t0` initial time
- `t` vector of times where the solution is computed (must start with `t0`)
- `f` function defining the system

Example — solving $y'' = -4y$, with $y(0)=3, y'(0)=0$:

scilab

```
function yprim=f(t,y)
    yprim(1) = y(2);
    yprim(2) = -4*y(1);
endfunction

t0 = 0; tmax = 5;
t = t0:0.05:tmax;
y0 = 3; yprim0 = 0;
y = ode([y0; yprim0], t0, t, f);
clf; plot(t, y(1,:))
```

The trick: let $y(1)$ be the function and $y(2)$ be its derivative, turning one 2nd-order ODE into two 1st-order equations.

Useful Functions Reference

Analysis

Function	Returns
$\text{sqrt}(x)$	square root
$\text{log}(x)$	natural log
$\text{exp}(x)$	exponential
$\text{abs}(x)$	absolute value (or modulus if complex)
$\text{int}(x)$	truncation toward zero
$\text{floor}(x)$	integer part ($\leq x$)
$\text{ceil}(x)$	rounds up
$\text{factorial}(n)$	$n!$

Vectors / statistics

Function	Returns
$\text{length}(v)$	number of elements in a vector

Function	Returns
<code>size(m)</code>	dimensions (rows, columns) of a matrix
<code>sum(v)</code> / <code>prod(v)</code>	sum / product of all elements
<code>cumsum(v)</code>	running cumulative totals
<code>mean(v)</code> / <code>stdev(v)</code>	average / standard deviation
<code>gsort(v)</code>	sort descending
<code>gsort(v,"g","i")</code>	sort ascending
<code>unique(v)</code>	remove duplicates, sort ascending
<code>find(v < 5)</code>	indices of elements passing a test
<code>zeros(n,p)</code>	n×p matrix of zeros
<code>feval(x,y,f)</code>	matrix of f evaluated over x,y

Random numbers

Function	Returns
<code>grand(1,p,"uin",m,n)</code>	p random integers between m and n
<code>grand(1,p,"unf",a,b)</code>	p random reals between a and b
<code>rand(n,p)</code>	n×p matrix of reals in [0,1]
<code>rand()</code>	one real in [0,1]

Display & utilities

Function	Does
<code>disp(x)</code>	display a value/matrix
<code>disp("text" + string(x))</code>	display text combined with a number
<code>string(x)</code>	convert a number to text
<code>format(n)</code>	set display to n digits (e.g. <code>format(20)</code> to see rounding error)

Function	Does
<code>help name</code>	open help for a function (e.g. <code>help sin</code>)
<code>tic</code> / <code>toc</code>	start / stop a timer
<code>xclick</code>	return coordinates of a click in a graphics window

Workflow Tips

- **Console** = quick calculations. **Editor (SciNotes)** = writing/saving programs.
- Save editor files with a `.sce` extension.
- Run a program: **Execute** menu — with echo (`Ctrl+L`) reprints it in the console, without echo (`Ctrl+Shift+E`) just runs it.
- Don't type the `-->` prompt in the editor — that's only the console marker shown in tutorials.
- Interrupt a running program: type `pause` or `Ctrl+X`; `resume` to continue, `abort` to quit entirely.
- `Ctrl+I` fixes indentation in the editor.